



RETO

Gestión Localización del Material del Taller de Informática.

Manual realizado por: Iván Gómez,

Gleb Nesterov, Hugo Andrea, Benjamín Rivero,

David Chisca y Lara Houela.

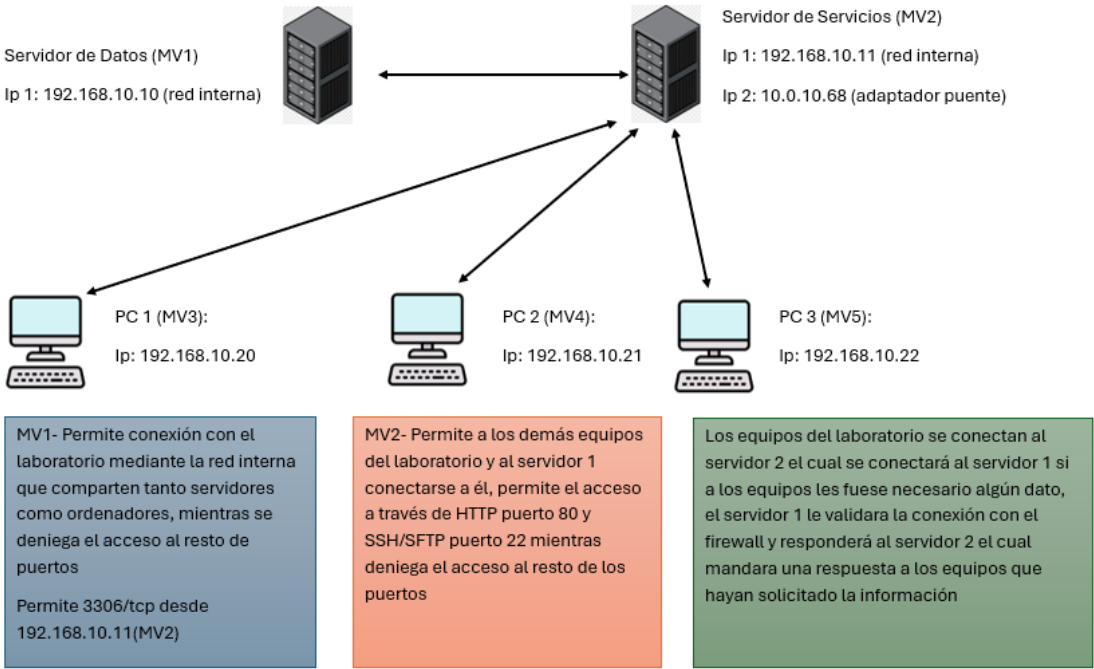
Equipo 2

DAW1

INDICE

Diagrama de arquitectura de red.	3
Comparativa de tecnologías probadas.....	4
Nuestra elección:	5
MV1. Servidor de datos.	6
1.1. Instalación de Ubuntu Server.....	6
1.2. Instalación de MySQL.	9
1.3. Configuración del firewall.....	10
1.4. Configuración de MySQL.	11
MV2. Servidor de servicios.	13
1.1. Instalación de Ubuntu Server.....	13
1.2. Instalación de apache.	17
COMPROBACIÓN EN EQUIPO CLIENTE.	19

Diagrama de arquitectura de red.



Comparativa de tecnologías probadas.

1. VirtualBox- Es un hipervisor de tipo 2 que emula hardware físico para ejecutar sistemas operativos completos sobre el ordenador anfitrión. Lo mejor de esta es el poder probar diferentes sistemas operativos para que no haya que instalarlos directamente en el ordenador anfitrión. Lo peor de esta es el arranque que suele ser lento debido a que tiene que iniciar un sistema operativo entero y los altos recursos que requiere cada máquina.
2. Docker- Es una plataforma de contenedores (actúa como un SO para entornos en la nube) a diferencia de las MV los contenedores comparten el kernel del SO anfitrión siendo más ligeros. Ideal para microservicios, aplicaciones modernas y despliegue rápido. Otras ventajas de este son el bajo consumo de recursos, arranque muy rápido en pocos segundos y también consta de una alta portabilidad. Por otro lado, unas de las desventajas de Docker son su seguridad que al compartir el núcleo con el anfitrión los contenedores no están tan aislados ya que si falla algo en el anfitrión haría que fallasen los contenedores, gestionar muchos contenedores diferentes puede ser complejo ya que la gestión de redes y almacenamiento de datos persistentes requieren mucho conocimiento, aunque los contenedores consuman pocos recursos si hay un contenedor mal configurado puede llegar a tal punto en el que puede consumir toda la CPU o memoria del host.
3. Vagrant- Este se diferencia respecto a los otros dos es que este no es un hipervisor como VirtualBox o Docker, sino que gestiona y automatiza MVs usando proveedores como VirtualBox, VMware o AWS para crear diferentes entornos de desarrollo. Lo mejor de este es que se pueden crear entornos de desarrollo idénticos entre sí para todo el equipo lo que significa que si a uno le funciona una cosa al resto también, otro punto a favor es que la configuración de la MV es bastante rápida a comparación de VirtualBox, pero más lenta que Docker. Por otra parte, los recursos sí que suelen ser altos al usar MVs, la complejidad de gestión en cuanto al mantenimiento de las cajas puede ser difícil, al igual que se depende de un hipervisor externo que también debe configurarse y puede ser complejo.

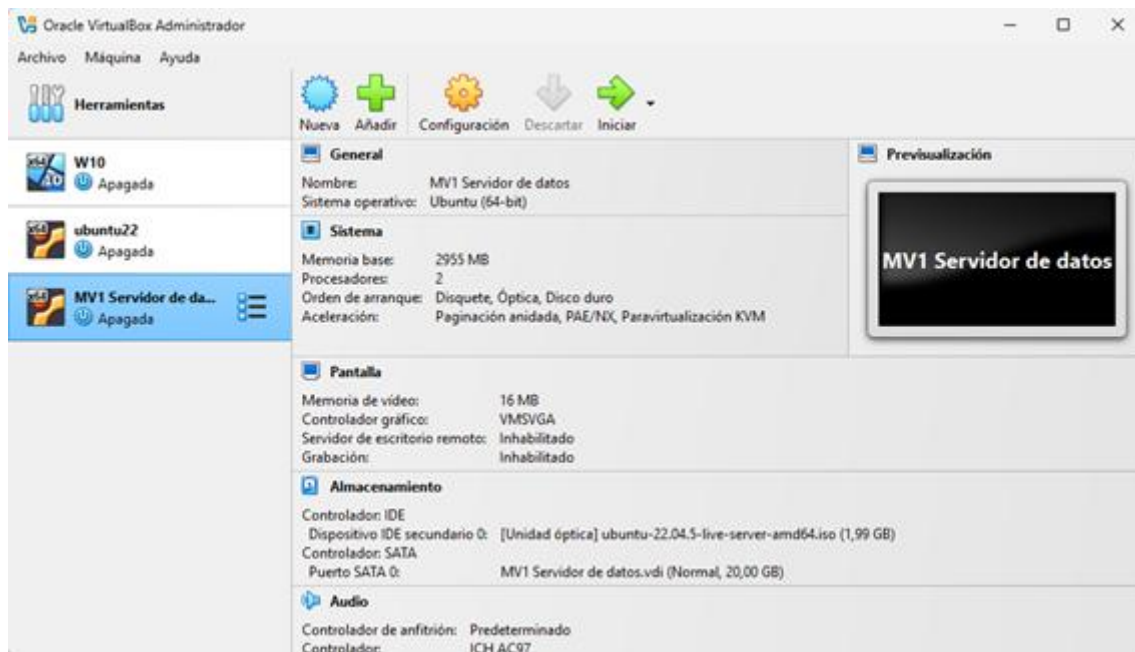
Nuestra elección:

En nuestro caso a la hora de realizar las MVs hemos decidido usar VirtualBox ya que es con la que hemos realizado todas las MVs durante el curso y nos parecía la más sencilla de usar con los conocimientos que hemos adquirido usándola, así todo viendo cómo funcionan Docker y Vagrant quizás podríamos haber usado Vagrant junto al hipervisor VirtualBox lo cual es muy común por personas que dominan el uso de ambas, aun así veo bien nuestra decisión ya que estas MVs que hemos creado ahora no van a durar tanto como para dedicarnos a usar Vagrant junto con el hipervisor elegido con anterioridad.

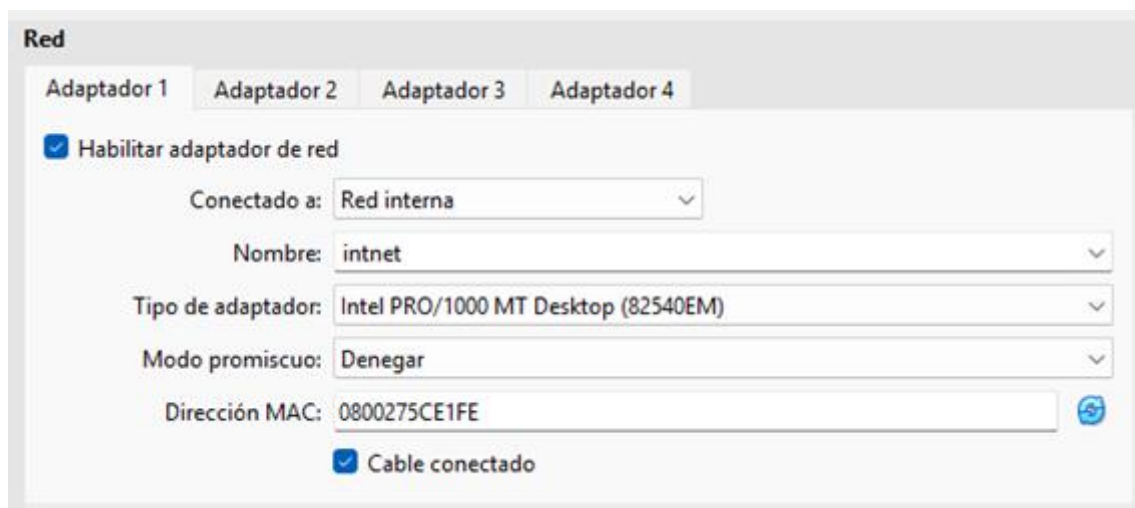
MV1. Servidor de datos.

1.1. Instalación de Ubuntu Server.

Creamos una máquina virtual Linux y añadimos la iso de Ubuntu server 22.04



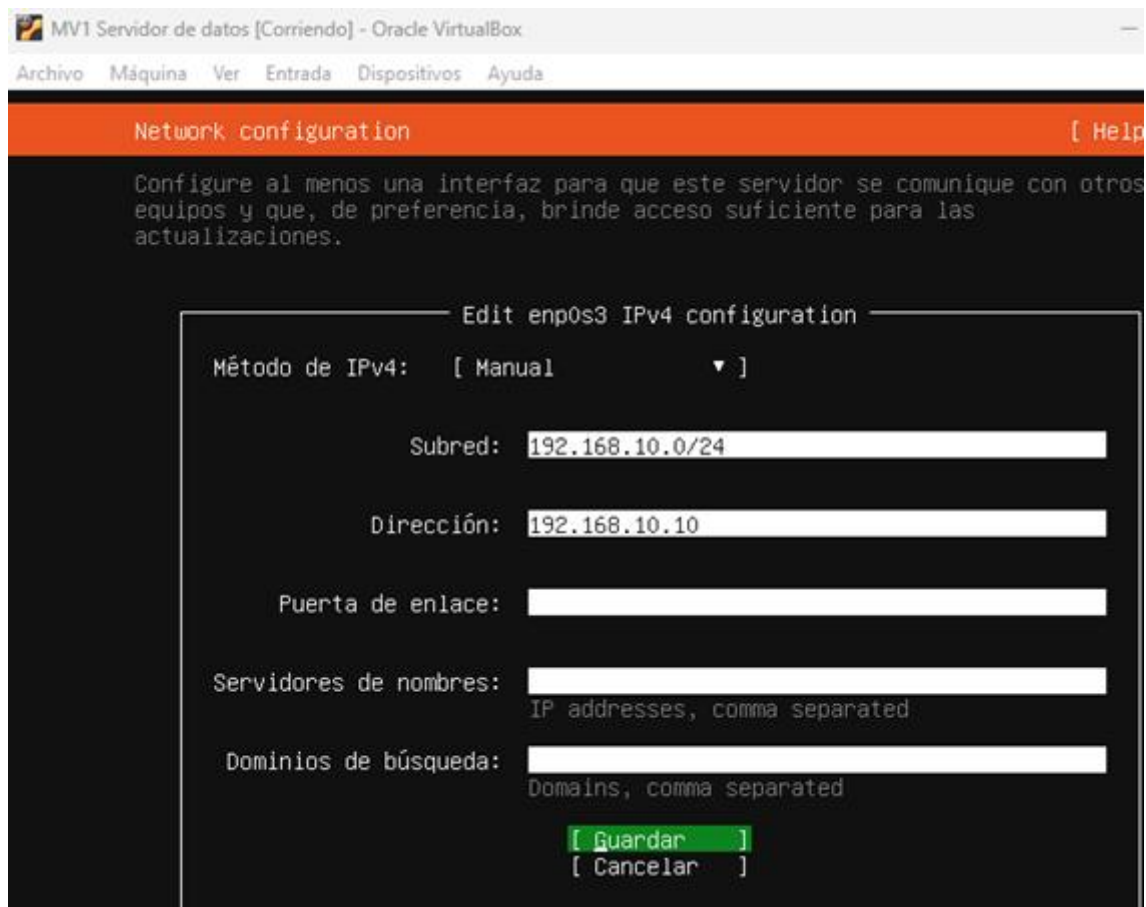
El adaptador de red tiene que ser red interna para poder conectarse desde la MV2 a la MV1 sin que ningún otro equipo interfiera desde el exterior.



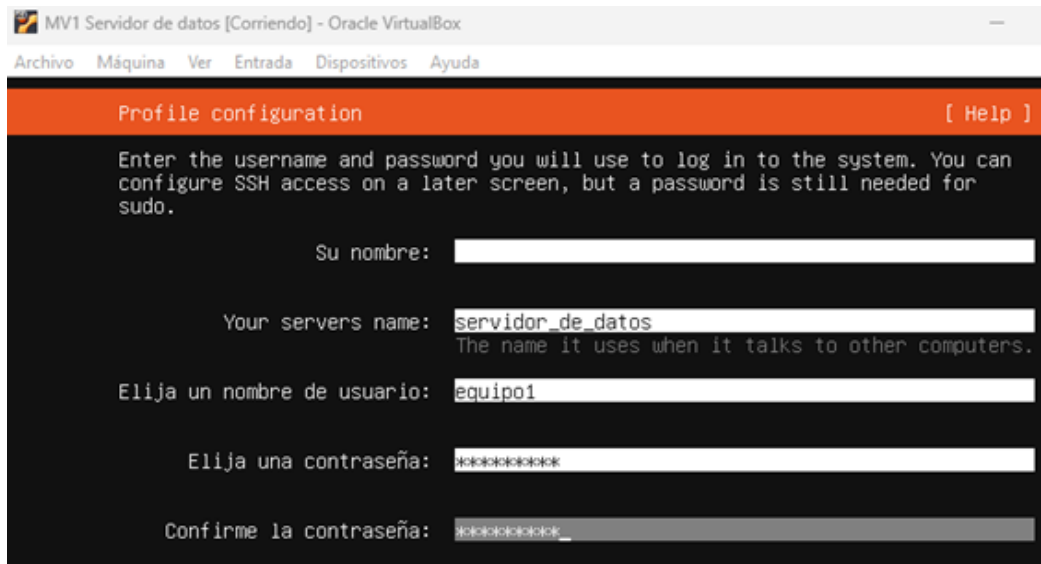
Vamos a instalar Ubuntu server base.



La configuración de la ip será la 192.168.10.10 . Como es red interna no hace falta puertos de enlace.

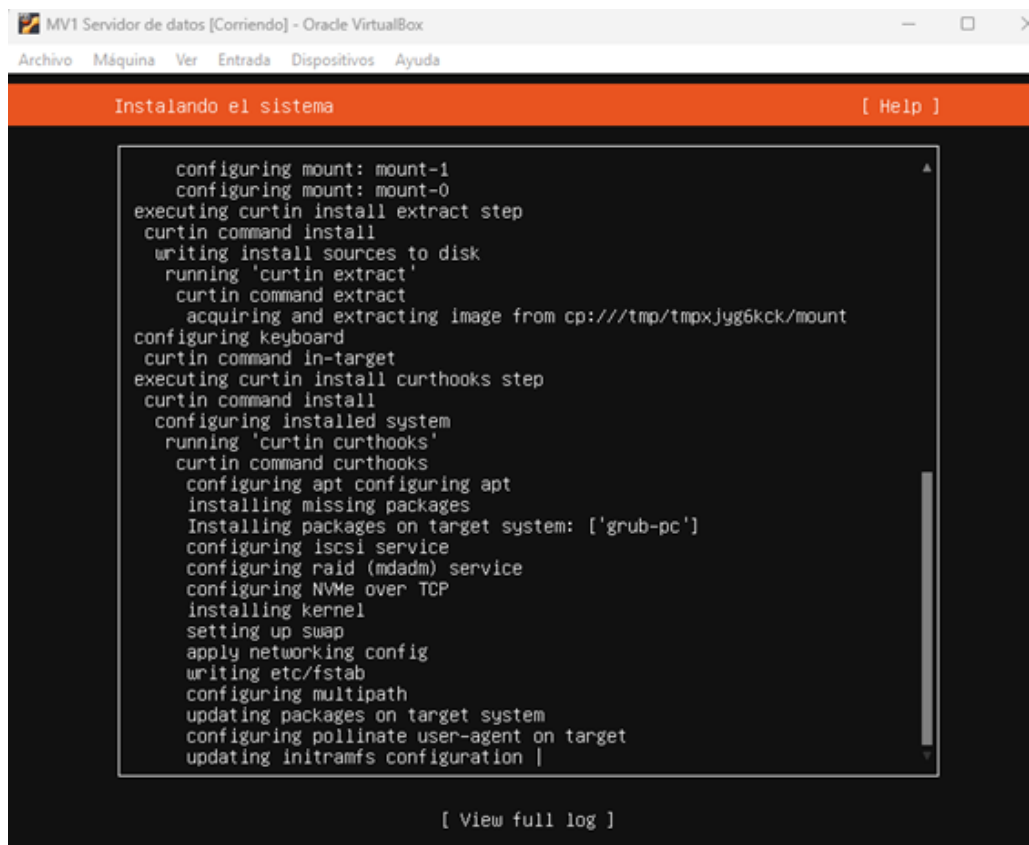


Los nombres para el servidor y el usuario (el nombre es equipo 1 debido a un error)



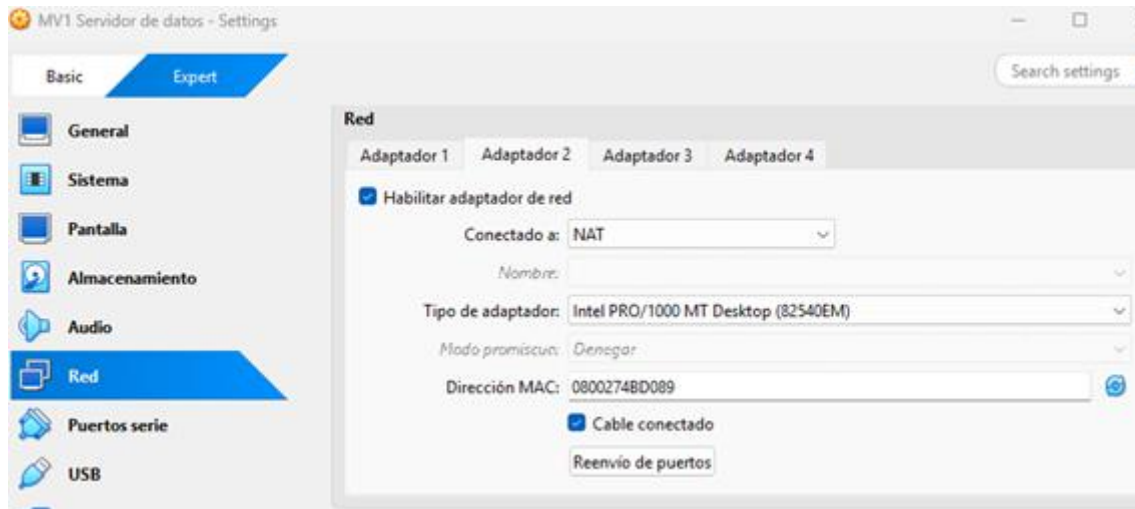
La contraseña es usuario@1

Y lo dejas instalando



1.2. Instalación de MySQL.

Para poder instalar MySQL en Ubuntu server vas a tener que añadir temporalmente una tarjeta de red NAT para poder conectarse a la red.



Para configurar la ip temporal de la red nat hacemos un `sudo dhclient`.

```
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
equipo1@servidordedatos:~$ sudo dhclient
```

Ahora necesitamos actualizar los paquetes de Ubuntu server.

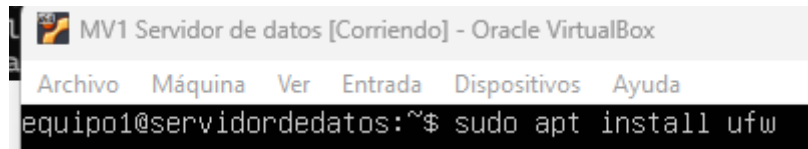
```
Last login: Wed May 6 13:22:39 UTC 2026 on tty1
equipo1@servidordedatos:~$ sudo apt update_
```

Ahora ya podemos instalar el MySQL.

```
MV1 Servidor de datos [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
equipo1@servidordedatos:~$ sudo apt install mysql-server
```

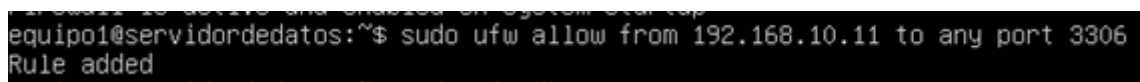
1.3. Configuración del firewall.

Para configurar el firewall tenemos que instalar ufw para poder editarlo si no lo tienes instalado ya.



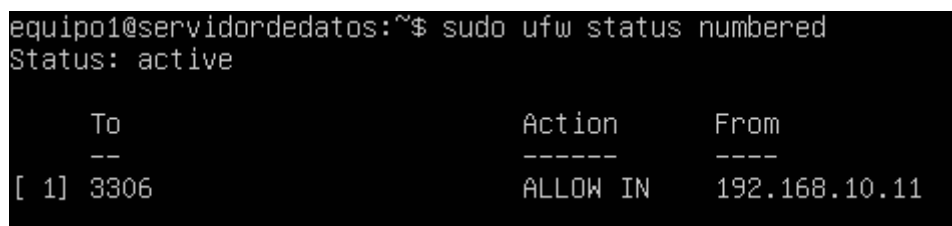
```
equipo1@servidordedatos:~$ sudo apt install ufw
```

La primera regla que vamos a añadir va a ser que la ip de la MV2 pueda acceder por el puerto 3306 de la MV1.



```
equipo1@servidordedatos:~$ sudo ufw allow from 192.168.10.11 to any port 3306
Rule added
```

Comprobamos que en las reglas de firewall este solo configurado la conexión de la MV2 mediante el puerto 3306.



```
equipo1@servidordedatos:~$ sudo ufw status numbered
Status: active

      To      Action      From
      --      -
[ 1] 3306     ALLOW IN    192.168.10.11
```

1.4. Configuración de MySQL.

Permitimos en el firewall el acceso al puerto 3306/tcp para que se pueda conectar la MV2.

```
equipo1@servidordedatos:~$ sudo ufw allow 3306/tcp_
```

Ahora vamos a configurar un archivo para permitir que todas las ip puedan acceder a MySQL.

```
equipo1@servidordedatos:~$ sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

En el archivo cambiamos el bind-address a 0.0.0.0 para permitir el acceso.

```
GNU nano 6.2 /etc/mysql/mysql.conf.d/mysqld.cnf
#
# The MySQL database server configuration file.
#
# One can use all long options that the program supports.
# Run program with --help to get a list of available options and with
# --print-defaults to see which it would actually understand and use.
#
# For explanations see
# http://dev.mysql.com/doc/mysql/en/server-system-variables.html
#
# Here is entries for some specific programs
# The following values assume you have at least 32M ram
#
[mysqld]
#
# * Basic Settings
#
user                = mysql
# pid-file           = /var/run/mysqld/mysqld.pid
# socket             = /var/run/mysqld/mysqld.sock
# port               = 3306
# datadir            = /var/lib/mysql

# If MySQL is running as a replication slave, this should be
# changed. Ref https://dev.mysql.com/doc/refman/8.0/en/server-system-variab
# tmpdir             = /tmp
#
# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address         = 0.0.0.0
mysqlx-bind-address = 127.0.0.1
```

Reiniciamos el servidor MySQL y comprobamos que está funcionando correctamente.

```
equipo1@servidordedatos:~$ sudo systemctl restart mysql_
```

```
equipo1@servidordedatos:~$ sudo systemctl status mysql
• mysql.service - MySQL Community Server
   Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-05-11 16:52:36 UTC; 1h 7min ago
     Process: 2269 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
    Main PID: 2277 (mysqld)
      Status: "Server is operational"
        Tasks: 38 (limit: 3280)
       Memory: 371.7M
          CPU: 1min 3.066s
      CGroup: /system.slice/mysql.service
             └─2277 /usr/sbin/mysqld
```

Comprobamos que está bien configurado si con el comando `ss -tulnp | grep 3306` y aparece `0.0.0.0:3306` está bien configurado, pero si aparece `127.0.0.1` significa que el archivo anterior está mal editado.

```
equipo1@servidordedatos:~$ sudo ss -tulnp | grep 3306
tcp LISTEN 0      70      127.0.0.1:33060  0.0.0.0:*    users:((("mysqld",pid=2277,fd=21))
tcp LISTEN 0      151     0.0.0.0:3306   0.0.0.0:*    users:((("mysqld",pid=2277,fd=23))
equipo1@servidordedatos:~$ _
```

Ahora vamos a configurar un usuario en mysql para poder entrar desde la MV2.

```
mysql> create user 'admin'@'192.168.10.11' identified by 'password';_
```

Le vamos a dar todos los permisos para que pueda actuar como administrador en todas las bases de datos.

```
mysql> grant all privileges on *.* to 'admin'@'192.168.10.11';
```

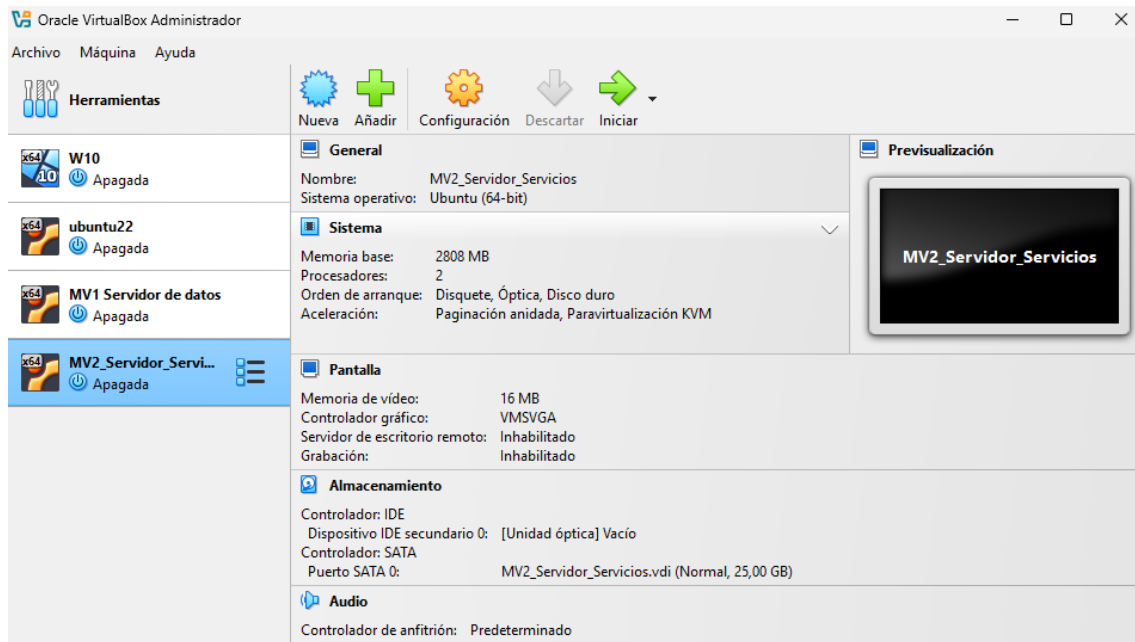
Y actualizamos los permisos.

```
mysql> flush privileges;
Query OK, 0 rows affected (0,01 sec)
```

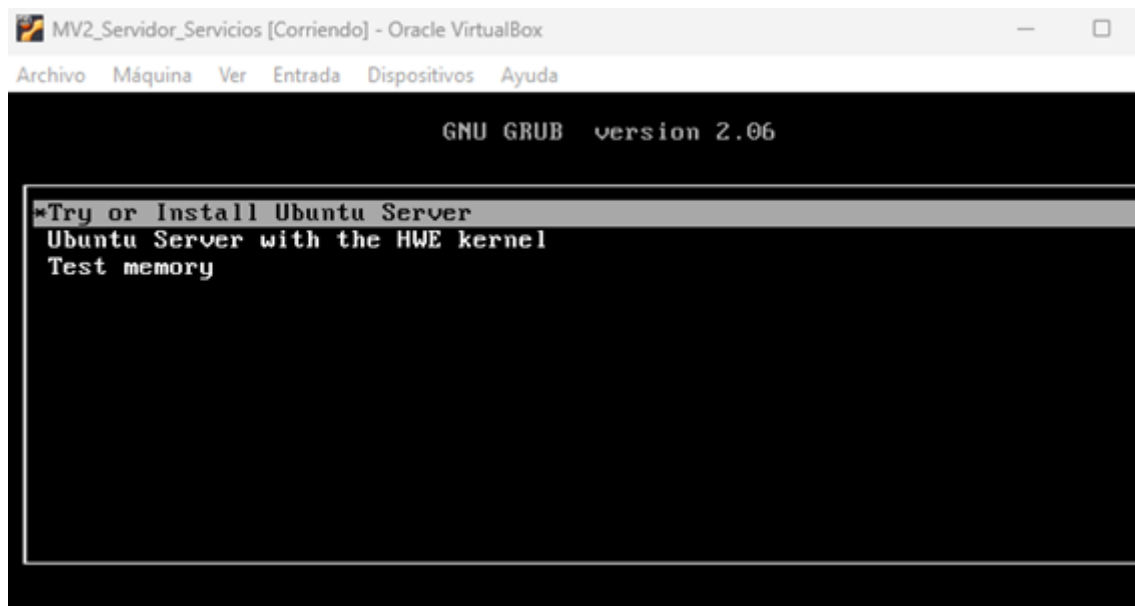
MV2. Servidor de servicios.

1.1. Instalación de Ubuntu Server.

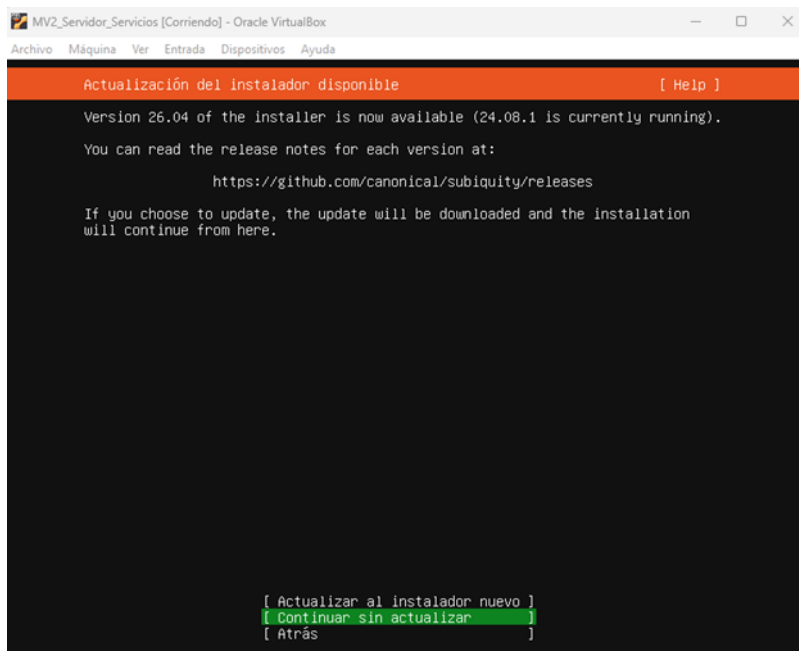
Vamos a crear una segunda máquina virtual con ubuntu server.



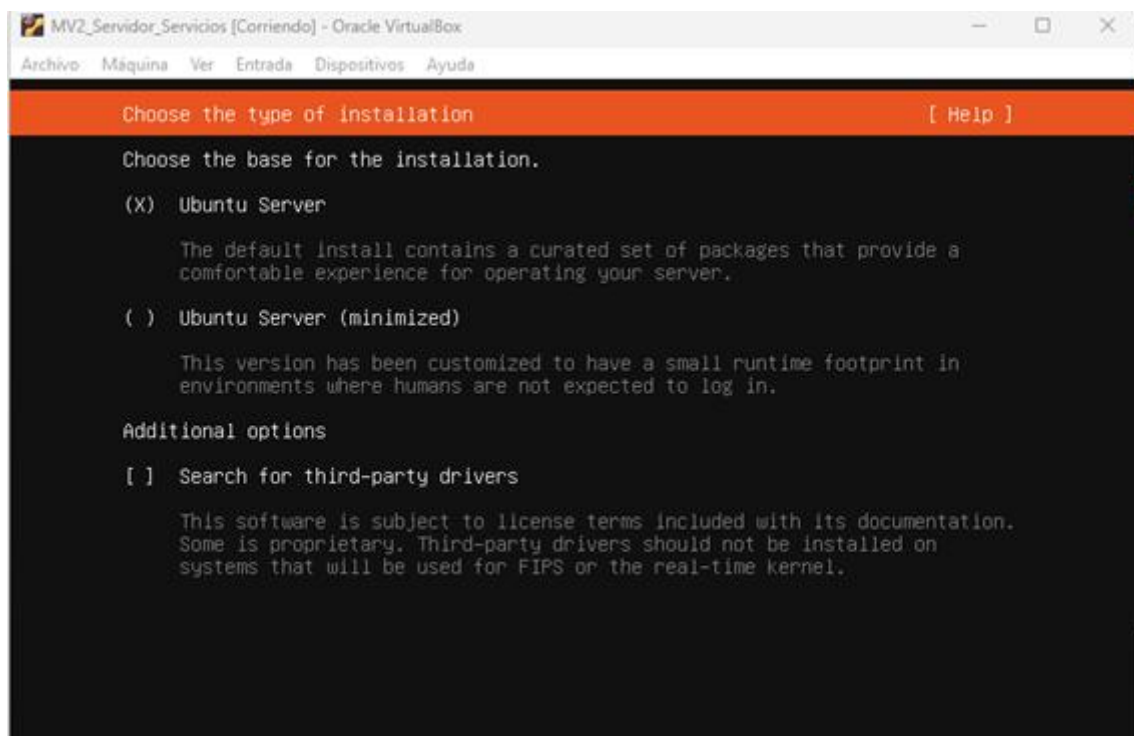
Instalamos ubuntu server base.



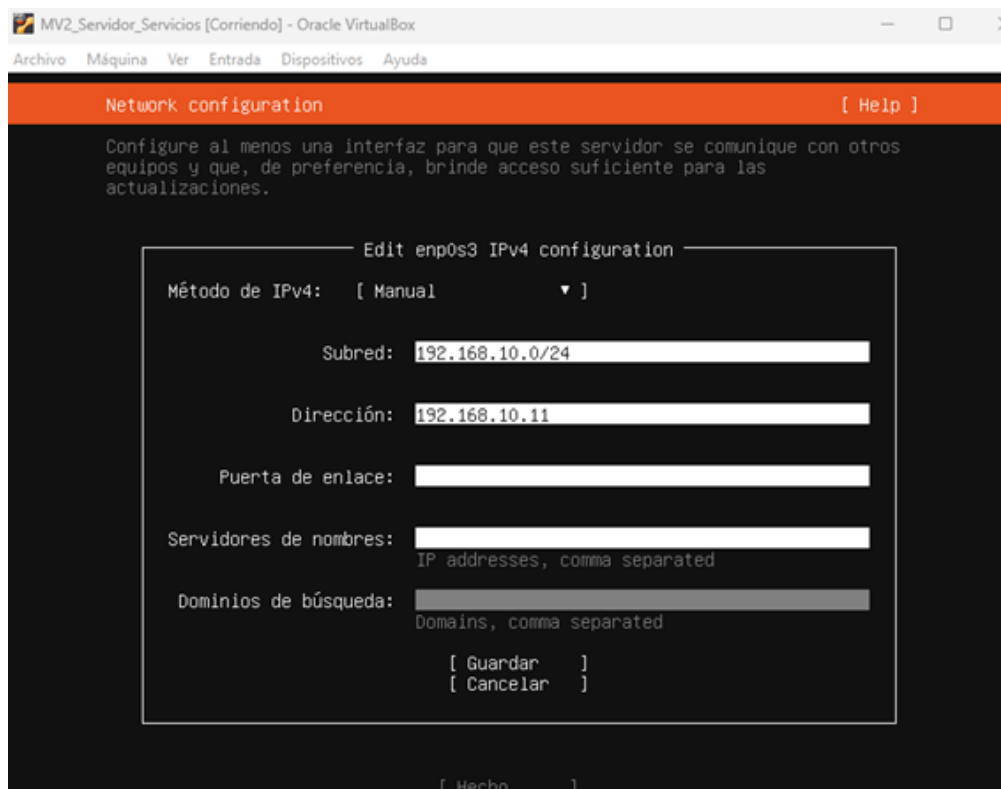
No es necesario actualizar el instalador ya que vamos a usar ubuntu 22.



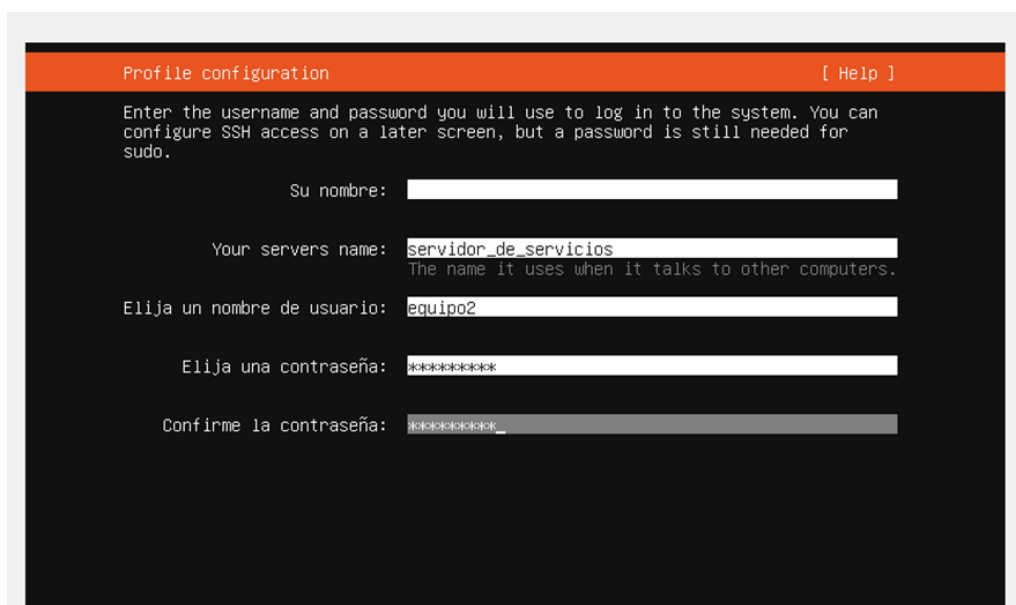
Seleccionamos el ubuntu server base.



La configuración del primer adaptador de red que es el de red interno va a tener la ip 192.168.10.11.



La configuración del perfil va a tener los siguientes parámetros el nombre del server va a ser servidor_de_servicios y el nombre de usuario equipo2 con la contraseña usuario@1.



Debemos marcar la opción del ssh que nos la pide para esta máquina virtual.

```
SSH configuration [ Help ]

You can choose to install the OpenSSH server package to enable secure remote
access to your server.

[ ] Instalar servidor OpenSSH

[X] Permitir autenticación con contraseña por SSH

[ Import SSH key ► ]

AUTHORIZED KEYS

No authorized key
```

Para configurar la ip que da al exterior va a ser estática y para eso va a tener que crear este archivo.

```
equipo2@servidordeservicios:~$ sudo nano /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg_
```

Y añadir `network: {config: disabled}`

```
GNU nano 6.2 /e
network: {config: disabled}
```

Luego accedemos a este archivo

```
equipo2@servidordeservicios:~$ sudo nano /etc/cloud/cloud.cfg.d/90-installer-network.cfg
```


Y ponemos la configuración de la ip estática

```
GNU nano 6.2 /etc/cloud/cloud.cfg.d/50-cloud-init.cfg
# This is the network config written by 'subiquity'
network:
  ethernets:
    enp0s3:
      addresses:
        - 192.168.10.11/24
      nameservers:
        addresses: []
        search: []
    enp0s8:
      addresses:
        - 10.0.10.23/24
      nameservers:
        addresses: []
        search: []
  version: 2
```

Una vez todo configurado entramos en la configuración de las ip.

```
equipo2@servidordeservicios:~$ sudo nano /etc/netplan/50-cloud-init.yaml
```

Y configuramos la segunda ip como estática.

```
GNU nano 6.2 /etc
# This file is generated from information
# to it will not persist across an instar
# network configuration capabilities, wri
# /etc/cloud/cloud.cfg.d/99-disable-netw
# network: {config: disabled}
network:
  ethernets:
    enp0s3:
      addresses:
        - 192.168.10.11/24
      nameservers:
        addresses: []
        search: []
    enp0s8:
      addresses:
        - 10.0.10.23/24
      nameservers:
        addresses: []
        search: []
  version: 2
```

1.2. Instalación de apache.

Antes de empezar la instalación de apache necesitamos actualizar los repositorios.

```
equipo2@servidordeservicios:~$ sudo apt update_
```

Una vez actualizados los paquetes de ubuntu se instalará el apache con el comando.

```
equipo2@servidordeservicios:~$ sudo apt install apache2
```

Comenzamos con la creación de un grupo para para que los usuarios solo puedan acceder al sftp

```
equipo2@servidordeservicios:~$ sudo groupadd sftpgroup
```

Creamos un usuario con distintos atributos -G se asigna al grupo declarado en el comando, -d determina el directorio de inicio y -s regula el acceso al Shell

```
equipo2@servidordeservicios:~$ sudo useradd -G sftpgroup -d /srv/sftpuser -s /sbin/nologin sftpuser
```

Creamos una carpeta para el sftpuser

```
equipo2@servidordeservicios:~$ sudo mkdir -p /srv/sftpuser
```

Asignamos la propiedad al usuario root

```
equipo2@servidordeservicios:~$ sudo chown root /srv/sftpuser
```

Asignamos derechos de lectura y ejecución al grupo

```
equipo2@servidordeservicios:~$ sudo chmod g+rx /srv/sftpuser
```

Creamos un subdirectorio y asignamos al usuario sftpuser como propietario

```
equipo2@servidordeservicios:~$ sudo mkdir -p /srv/sftpuser/data
```

COMPROBACIÓN EN EQUIPO CLIENTE.

Ahora desde un equipo cliente vamos a probar la conexión al servidor de servicios y de hay al servidor de datos.

Desde el terminal del cliente ejecutamos el comando `ssh usuario@ip_server`

```
D:\Usuarios\daw129>ssh equipo2@10.0.10.23
```

Una vez dentro vamos a hacer `mysql -h IP_SERVIDOR -u USUARIO -p` para conectarse al msql del servidor.

```
equipo2@servidordeservicios:~$ mysql -h 192.168.10.10 -u admin -p
Enter password:
```

Si todo ha funcionado correctamente tendrá que estar en la minea de código con `mysql`.

```
mysql>
```